Ensemble Learning – Research Project (2026/2027)

Instructor: Stanisław Kaźmierczak

Sequential Functional Orthogonality via Triangular Stop-Gradient Masks in Tabular ResNet Ensembles

Initial Plan

Author 1:
Yahor Kachanouski
Student ID: 329982

Author 2:
Antoni Kingston
Student ID: 327284

October 27, 2026

Contents

1 Introduction and Research Problem **2**

2 Literature Review **2**

3 Proposed Method and Validation Procedure **3**
3.1 Proposed Architecture and Methodology 3
3.2 Validation Procedure 5
3.3 Evaluation Metrics 5

4 Hyperparameters and Tuning Strategy **6**
4.1 Planned Algorithm Hyperparameters 6
4.2 Selection and Tuning Strategy 6

5 Datasets and Preprocessing **7**
5.1 Classification Datasets 7
5.2 Regression Datasets 7
5.3 Preprocessing Procedures 9

6 Results Aggregation and Statistical Analysis **9**
6.1 Approach to Aggregating Results 9
6.2 Planned Statistical Tests 10

7 Reproducibility and Declarations **11**
7.1 Reproducibility 11
7.2 External Code and AI Usage Disclosure 11

1 Introduction and Research Problem

Combining multiple models in an ensemble is a primary mechanism for reducing variance and improving generalization in machine learning. While tree-based ensembles have historically dominated structured tabular benchmarks [7], well-regularized Tabular Neural Networks (TNNs)—such as TabNet [1], well-tuned dense networks [11], and Tabular ResNets [7]—have narrowed this performance gap, particularly when deployed as ensembles.

Across deep learning domains, training an ensemble of neural networks presents a fundamental challenge in structuring how constituent models partition the feature space:

1. **Unregularized Training:** In general neural ensembles, training multiple networks without explicit diversity constraints relies solely on random weight initialization to explore distinct local minima in the loss landscape [5]. Because no mechanism actively penalizes representational overlap, independently initialized networks frequently converge toward redundant latent features. This general phenomenon becomes particularly pronounced when applying TNNs to tabular datasets, where a small subset of informative features often dominates the target variance [7].

2. **Symmetric Repulsion:** Methods such as Negative Correlation Learning (NCL) [14] and Orthogonal Ensemble Networks [13] introduce symmetric pairwise similarity penalties across models. However, because symmetric penalties apply bidirectional gradients to all models simultaneously, no single model is permitted to greedily capture the primary predictive representation, and learners may collude by introducing mutually canceling biases to satisfy the penalty [10].

To resolve this issue, this project investigates **Sequential Functional Orthogonality via a Triangular Stop-Gradient Mask (SFOvTSGM)** for additive ensembles of Tabular ResNet models. By applying a strictly lower-triangular similarity penalty coupled with a stop-gradient operator (sg) to the batch-centered latent representations, we impose an asymmetric hierarchy ($1 < 2 < \cdots < M$) during simultaneous training. The primary network ($m = 1$) is exempted from the orthogonality penalty, allowing it to capture the dominant data structure undisturbed. Each subsequent network ($m > 1$) contributes to the ensemble prediction while being repelled from the detached latent representations of all preceding networks ($j < m$).

2 Literature Review

Ensemble Objectives and Representation Redundancy. The predictive advantage of an ensemble depends on balancing individual model capability with inter-model diversity. Early neural ensemble frameworks such as Negative Correlation Learning (NCL) [14] modified the training objective to decorrelate errors across models. Later, Fort et al. [5] demonstrated that training separate neural networks from different random initializations guides them into distinct basins of attraction in the loss landscape. To reduce the memory overhead of explicit ensembles, Wen et al. [15] introduced BatchEnsemble, which parameterizes ensemble members via rank-1 scaling adapters over a shared weight matrix. However, subsequent theoretical and empirical work by Jeffares et al. [10] showed that naively optimizing joint ensemble losses or symmetric diversity penalties leads to learner collusion, where individual networks shift their representations in opposing directions that cancel out upon aggregation without improving test generalization.

Latent Geometry and Asymmetric Orthogonalization. Quantifying or penalizing redundancy in hidden layers requires similarity metrics invariant to neuron permutations and orthogonal transformations. Kornblith et al. [12] introduced Centered Kernel Alignment (CKA), which compares the geometric structures of two latent spaces via their centered pairwise Gram matrices. In computer vision, Larrazabal et al. [13] showed that applying orthogonality constraints

to latent representations improves ensemble performance, though their formulation relied on symmetric pairwise repulsion.

To enforce ordered orthogonality without perturbing preceding representations, asymmetric stop-gradient operators have been developed in adjacent fields:

- **Spectral Decomposition:** Gemp et al. [6] and Deng et al. [4] formulated a simultaneous analog to Gram–Schmidt orthogonalization for learning ordered eigenfunctions. By applying a strictly lower-triangular penalty where function m orthogonalizes against $\mathrm{sg}(f_j)$ for $j < m$, the top function converges to the principal component undisturbed while higher-index functions span orthogonal complements in parallel.
- **Sequential Representation Repulsion:** In medical anomaly detection, Gu et al. [8] trained autoencoder ensembles sequentially, penalizing the CKA similarity between the active learner and previously frozen learners to prevent representation collapse.

Tabular Neural Networks and Regularization. Because tabular data consists of heterogeneous columns lacking spatial or sequential locality, neural networks are highly susceptible to overfitting uninformative features [7]. Arik and Pfister [1] introduced TabNet, utilizing sequential attention for instance-wise feature selection. Conversely, Kadra et al. [11] and Grinsztajn et al. [7] demonstrated that dense architectures—specifically Pre-Activation Tabular ResNets equipped with residual skip connections, batch normalization, and heavy dropout—provide a strong, computationally efficient inductive bias for tabular manifolds. To further regularize tabular representations, Jeffares et al. [9] proposed TANGOS, which enforces symmetric gradient orthogonality among neurons within a single network to encourage latent specialization.

3 Proposed Method and Validation Procedure

While intra-model orthogonality [9] restricts synergistic neuron co-adaptation within a single network, unregularized tabular ensembles [5, 7, 15] do not explicitly prevent constituent networks from learning redundant latent representations. Combining the simultaneous lower-triangular stop-gradient operator [4, 6] with latent geometric repulsion [8, 12] provides a mechanism to enforce ordered inter-model specialization across Tabular ResNet ensembles while preserving internal feature co-adaptation.

3.1 Proposed Architecture and Methodology

Tabular ResNet Backbone. We consider an ensemble $\mathcal{E} = \{f_1, f_2, \ldots, f_M\}$ of M separate Tabular ResNet models [7] with independent parameters $\theta_1, \ldots, \theta_M$. Each network maps an input vector of numerical and encoded categorical features into a d-dimensional hidden space via an initial linear projection stem, followed by L pre-activation residual blocks. Each residual block updates its intermediate representation $u \in \mathbb{R}^d$ according to:

$$\mathrm{ResBlock}(u) = u + \mathrm{Drop}_2\Big(W_2\,\mathrm{Drop}_1\big(\mathrm{ReLU}(W_1\,\mathrm{BN}(u) + b_1)\big) + b_2\Big), \tag{1}$$

where $W_1, W_2 \in \mathbb{R}^{d\times d}$ and $b_1, b_2 \in \mathbb{R}^d$ are learnable weights and biases, $\mathrm{BN}(\cdot)$ denotes Batch Normalization, and $\mathrm{Drop}_1, \mathrm{Drop}_2$ denote Dropout layers. Unlike vision ResNets, dense linear layers enable global feature mixing at every block, while dual dropout regularizes against tabular overfitting. After the final residual block, the normalized activation matrix $Z_m = \mathrm{ReLU}(\mathrm{BN}(U_m^{(L)})) \in \mathbb{R}^{B\times d}$ across a mini-batch of size B serves as the penultimate latent representation prior to the linear prediction head.

Additive Ensemble Prediction. Given an input mini-batch $X \in \mathbb{R}^{B\times D}$ with target Y, each Tabular ResNet $m \in \{1,\dots,M\}$ computes its penultimate latent matrix $Z_m \in \mathbb{R}^{B\times d}$ and functional output $f_m(X)$ (raw logits in classification or continuous predictions in regression). Rather than uniform averaging, the inference-time prediction $\hat{Y}$ of the ensemble is formed as an additive expansion:

$$\hat{Y} = f_1(X) + \nu \sum_{m=2}^{M} f_m(X), \tag{2}$$

where $f_1(X)$ acts as the unregularized base predictor and $\nu \in (0,1]$ is a shrinkage coefficient scaling the contributions of subsequent orthogonalized learners. The basic training approach will investigate what we term **dynamic boosting**: each subsequent model learns to predict the residual of the ensemble formed by its predecessors (or an analogous task-appropriate error signal), with these residuals recalculated as the preceding models' predictions change. Thus, a model can learn from the current residual even while its predecessors are still training, rather than waiting for them to converge before fitting the next model. If time and resources allow, we will also research classical sequential boosting methods and compare them with an alternative in which every model predicts the same target Y independently.

Sequential Functional Orthogonality (SFOvTSGM) Objective. Let $H = I_B - \frac{1}{B}\mathbf{1}\mathbf{1}^\top$ be the batch-centering matrix, such that $\tilde{Z}_m = HZ_m = Z_m - \frac{1}{B}\mathbf{1}\mathbf{1}^\top Z_m$ denotes the zero-mean latent representation matrix across the mini-batch. To impose a sequential hierarchy $1 < 2 < \cdots < M$ during simultaneous training, we define the repulsion penalty for network m using a strictly lower-triangular sum over batch-centered, stop-gradient (sg) detached representations:

$$\mathcal{L}_{\text{SFO}}^{(m)} = \sum_{j=1}^{m-1} \Phi\Big(HZ_m,\, \text{sg}(HZ_j)\Big) = \sum_{j=1}^{m-1} \Phi\Big(\tilde{Z}_m,\, \text{sg}(\tilde{Z}_j)\Big), \quad \text{with } \mathcal{L}_{\text{SFO}}^{(1)} = 0, \tag{3}$$

where $\frac{\partial\, \text{sg}(\tilde{Z}_j)}{\partial \theta_j} = 0$ and $\Phi(\cdot,\cdot) \in [0,1]$ is a differentiable similarity measure.

The total loss function is composed of the task prediction loss of the entire ensemble and the normalized sequential orthogonalization penalty:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}}\big(\hat{Y}, Y\big) + \lambda \sum_{m=2}^{M} \frac{1}{m-1} \mathcal{L}_{\text{SFO}}^{(m)}, \tag{4}$$

where $\mathcal{L}_{\text{task}}$ is Cross-Entropy or Mean Squared Error (MSE), and $\lambda > 0$ controls the orthogonalization strength. Because no repulsion gradients flow into θ_1, the first network greedily captures the primary functional relationship, while each network $m > 1$ learns complementary residual features within the latent subspace orthogonal to $\{Z_1, \dots, Z_{m-1}\}$.

Similarity Measures (Φ). We will evaluate and compare two formulations of $\Phi\big(\tilde{Z}_m, \text{sg}(\tilde{Z}_j)\big)$:

1. **Linear Centered Kernel Alignment (CKA) [12]:** CKA measures the holistic geometric similarity between two latent spaces by comparing their inter-sample similarity structures (Gram matrices $K_m = \tilde{Z}_m \tilde{Z}_m^\top \in \mathbb{R}^{B\times B}$). By normalizing the Hilbert–Schmidt Independence Criterion (HSIC), Linear CKA can be computed in feature space in $\mathcal{O}(Bd^2)$ time:

$$\Phi_{\text{CKA}}\big(\tilde{Z}_m, \text{sg}(\tilde{Z}_j)\big) = \frac{\|\tilde{Z}_m^\top \text{sg}(\tilde{Z}_j)\|_F^2}{\|\tilde{Z}_m^\top \tilde{Z}_m\|_F \, \|\text{sg}(\tilde{Z}_j)^\top \text{sg}(\tilde{Z}_j)\|_F + \epsilon}. \tag{5}$$

   Linear CKA is invariant to orthogonal transformations (rotations, reflections, and neuron permutations) and isotropic scaling. Minimizing Φ_{CKA} forces network m to alter the pairwise distance geometry among data points relative to networks $1 \dots m-1$.

2. **Squared Functional Cosine Similarity:** Whereas CKA compares full d-dimensional relational geometry, Squared Functional Cosine Similarity enforces direct coordinate-wise orthogonality between aligned latent dimensions across the batch dimension. Treating each column $\tilde{z}_{m,k} \in \mathbb{R}^B$ ($k \in \{1, \ldots, d\}$) as a function evaluated on the B batch samples, the metric computes the mean squared cosine of the angle between corresponding feature trajectories:

$$\Phi_{\text{Cos}}\big(\tilde{Z}_m, \text{sg}(\tilde{Z}_j)\big) = \frac{1}{d} \sum_{k=1}^{d} \left(\frac{\tilde{z}_{m,k}^\top \, \text{sg}(\tilde{z}_{j,k})}{\|\tilde{z}_{m,k}\|_2 \, \|\text{sg}(\tilde{z}_{j,k})\|_2 + \epsilon} \right)^2 . \tag{6}$$

Computed in $\mathcal{O}(Bd)$ time, minimizing Φ_{Cos} acts as a direct batch-wise Gram–Schmidt projection, forcing the k-th latent unit of model m to be uncorrelated with the k-th latent unit of preceding models across the input distribution.

Baselines and Ablations. Using the Tabular ResNet architecture, we will compare six configurations: (1) Single Tabular ResNet, (2) Single Tabular ResNet + TANGOS [9], (3) Tabular ResNet + BatchEnsemble [15] (parameter-efficient shared-weight baseline), (4) Unregularized Tabular ResNet Ensemble ($\lambda = 0$), (5) Symmetric Orthogonal Tabular ResNet Ensemble (bidirectional repulsion without stop-gradient), and (6) Tabular ResNet Ensemble + SFOvTSGM (evaluated with both Φ_{CKA} and Φ_{Cos}).

3.2 Validation Procedure

All experiments will use 5-fold cross-validation (stratified for classification, standard for regression) for outer evaluation. In each outer iteration, one fold (20% of the data) will be held out exclusively for final evaluation. The remaining 80% will be split once into an inner training partition (64% of the full dataset) and an inner validation partition (16% of the full dataset). The same split will be used for all methods in that outer iteration. Hyperparameters will be selected using this single validation partition; no inner cross-validation will be performed. After selection, the chosen model will be refit on the full outer training partition and evaluated on its untouched outer test fold. This process will be repeated independently for all five outer folds. Each outer-fold refit will use 5 fixed random seeds; we will report metrics averaged across seeds and outer folds, along with their standard deviations.

3.3 Evaluation Metrics

- **Classification:** Accuracy, Area Under the ROC Curve (ROC-AUC), and Negative Log-Likelihood (NLL).
- **Regression:** Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2 score.
- **Ensemble Diagnostics:** (i) Staged cumulative test performance $\hat{Y}^{(m)} = f_1(X) + \nu \sum_{j=2}^{m} f_j(X)$ as a function of ensemble depth $m \in \{1, \ldots, M\}$ to quantify the marginal contribution of each successive learner, (ii) functional norm profiles $\|f_m(X)\|_2$ across index m, and (iii) post-training pairwise CKA and cosine similarity matrices across the M networks.
- **Hardware-Independent Computational Cost:** To evaluate efficiency independent of hardware or software implementation details, we will report: (i) **Floating-Point Operations (FLOPs)** per training step (accounting for both the forward/backward pass of the M networks and the $\mathcal{O}(M^2Bd^2)$ or $\mathcal{O}(M^2Bd)$ overhead of Φ_{CKA} and Φ_{Cos}), (ii) **Total FLOPs to Convergence** (FLOPs per step multiplied by the number of steps to early stopping), and (iii) **Total Trainable Parameters**.

4 Hyperparameters and Tuning Strategy

4.1 Planned Algorithm Hyperparameters

The hyperparameters are partitioned into three categories: (1) shared Tabular ResNet backbone and optimization parameters, (2) ensemble and SFOvTSGM parameters, and (3) baseline-specific regularization parameters. Table 1 summarizes the planned search spaces and default values.

Table 1: Hyperparameter search spaces and fixed defaults across evaluated models.

Category	Hyperparameter	Search Space / Values	Distribution / Ty
Backbone (Tabular ResNet)	Residual blocks (L)	$\{2, 3, 4\}$	Discrete
	Hidden dimension (d)	$\{64, 128, 256\}$	Discrete
	Dropout rate (p_{drop})	$[0.0, 0.5]$	Uniform
	Categorical embedding dim.	$\{8, 16, 32\}$	Discrete
Optimization	Optimizer	AdamW	Fixed
	Learning rate (η)	$[10^{-4}, 5 \times 10^{-3}]$	Log-uniform
	Weight decay (γ)	$[10^{-6}, 10^{-2}]$	Log-uniform
	Batch size (B)	$\{128, 256, 512\}$	Dataset-dependen
	Max epochs / Patience	200 / 16	Fixed
Ensemble & SFOvTSGM	Ensemble size (M)	$\{1, 2, \ldots, 8\}$	Fixed (Grid)
	Shrinkage coefficient (ν)	$[0.05, 1.0]$	Log-uniform
	Orthogonality weight (λ)	$[10^{-3}, 10^{1}]$	Log-uniform
	Similarity metric (Φ)	$\{\Phi_{\text{CKA}}, \Phi_{\text{Cos}}\}$	Categorical
Baselines (TANGOS / BatchEns.)	Symm. Ortho. weight (λ_{sym})	$[10^{-3}, 10^{1}]$	Log-uniform
	TANGOS sparsity (λ_1)	$[10^{-4}, 10^{-1}]$	Log-uniform
	TANGOS orthogonality (λ_2)	$[10^{-3}, 10^{0}]$	Log-uniform
	BatchEnsemble sub-models (M)	8	Fixed

Batch Size Selection (B). Because both Φ_{CKA} and Φ_{Cos} estimate empirical covariance and directional alignment across mini-batch samples, batch size B directly affects the variance of the orthogonality gradient. To avoid rank-deficient Gram matrices when $B < d$, we will enforce $B \geq d$ by setting $B = 256$ for standard datasets ($N \leq 50$k) and $B = 512$ for larger datasets, falling back to $B = 128$ (with $d \leq 128$) only on small datasets ($N < 2$k).

4.2 Selection and Tuning Strategy

Tuning both architectural dimensions (L, d) and method-specific regularization hyperparameters simultaneously for every ensemble configuration across 24 datasets is computationally prohibitive and introduces confounding capacity differences between baselines. Following established tabular benchmarking protocols [7, 11], we will employ a controlled **Two-Stage Bayesian Optimization Protocol** implemented via `Optuna` using the Tree-structured Parzen Estimator (TPE) sampler:

1. **Stage 1 (Backbone Architecture Calibration):** Within each outer fold, we calibrate the single unregularized Tabular ResNet over 40 TPE trials using that fold's single inner training/validation split to determine the optimal structural configuration $(L^*, d^*, p^*_{\text{drop}})$. The selected architecture is then held fixed across all subsequent methods within that outer fold, so that they share an identical per-learner parameter count and representational capacity.

2. **Stage 2 (Method-Specific Hyperparameter Tuning):** With the backbone architecture fixed to $(L^*, d^*, p^*_{\text{drop}})$, each method tunes its specific regularization and optimization parameters over 30 TPE trials per outer fold. Each trial is evaluated on that same single inner validation partition, using its primary validation loss as the objective:

   - *SFOvTSGM Ensembles:* Tune $(\lambda, \nu, \eta, \gamma)$ separately for Φ_{CKA} and Φ_{Cos}.
   - *Symmetric Orthogonal Ensemble:* Tune $(\lambda_{\text{sym}}, \eta, \gamma)$.
   - *Unregularized Ensemble & BatchEnsemble:* Tune (η, γ) with $\lambda = 0$.
   - *Single ResNet + TANGOS:* Tune $(\lambda_1, \lambda_2, \eta, \gamma)$.

Pruning and Early Stopping. For each tuning trial, training will be monitored on the single inner validation partition using the primary task loss (Cross-Entropy for classification, MSE for regression), excluding the regularization penalty term to ensure fair comparison across values of λ. Early stopping will be triggered if validation loss does not improve for 16 consecutive epochs, and `Optuna`'s `MedianPruner` (with a 10-epoch warmup) will terminate unpromising trials early. After selecting hyperparameters, the chosen model will be refit on the full outer training partition. The refit epoch count will be set using the best epoch observed on the inner validation split for the selected configuration. The outer test fold will not be used for hyperparameter selection, pruning, early stopping, or any other model-selection decision. All selection and refitting steps will be repeated independently for each outer fold.

5 Datasets and Preprocessing

To satisfy the requirement of evaluating at least 12 classification and 12 regression tasks while minimizing redundant hyperparameter tuning compute, all 24 datasets are sourced directly from the standardized **Inria Tabular Benchmark** curated by Grinsztajn et al. [7] (hosted on OpenML under Benchmark Suites 334, 335, 336, and 337, and on Hugging Face at `inria-soda/tabular-benchma` with reference code in the `LeoGrin/tabular-benchmark` repository), alongside overlapping datasets evaluated in the Cambridge `vanderschaarlab/TANGOS` repository [9].

This selection provides two practical advantages:

1. **Standardized Benchmark Context:** The published benchmark protocols and implementations from Grinsztajn et al. [7] and Jeffares et al. [9] provide reproducible reference points for our methods. Published hyperparameter logs or checkpoints will not be used to select or evaluate configurations; all tuning will use only the corresponding outer training partition and its single inner validation split.

2. **Controlled Sample Sizes:** We select datasets within $N \in$ [4,177, 98,049] to keep 5-fold outer cross-validation and 5 random seeds for each outer-fold refit computationally tractable on a single GPU, while ensuring every training fold contains enough samples to support batch sizes $B \in \{256, 512\}$ required for low-variance batch-wise Φ_{CKA} estimation.

5.1 Classification Datasets

Table 2 summarizes the 12 tabular classification datasets from the Inria OpenML benchmark suites (334 and 337) [7].

5.2 Regression Datasets

Table 3 summarizes the 12 tabular regression datasets from the Inria OpenML benchmark suites (335 and 336) [7].

Table 2: Summary of the 12 benchmark tabular classification datasets (OpenML Suites 334 & 337).

Dataset Name	**OpenML ID / Suite**	**Samples (N)**	**# Num**	**# Cat/Bin**	**Classes**	**Batch (B)**
Electricity	ID 44156 [7]	38,474	7	1	2	256
Eye Movements	ID 44157 [7]	7,608	20	3	2	256
MagicTelescope	ID 44125 [7]	13,376	10	0	2	256
Bank Marketing	ID 44126 [7]	10,578	7	0	2	256
Credit	ID 44089 [7]	16,714	10	0	2	256
Default Credit Card	ID 44124 [7]	13,272	20	1	2	256
Jannis	ID 44131 [7, 9]	57,580	54	0	2	512
Higgs (Subsampled)	ID 44129 [7, 9]	94,016	24	0	2	512
Pol	ID 44122 [7]	10,082	26	0	2	256
MiniBooNE	ID 44128 [7]	72,998	50	0	2	512
Compass	ID 44162 [7]	16,644	9	8	2	256
Covertype (Inria)	ID 44159 [7]	42,368	10	44	2	512

Table 3: Summary of the 12 benchmark tabular regression datasets (OpenML Suites 335 & 336).

Dataset Name	**OpenML ID / Suite**	**Samples (N)**	**# Num**	**# Cat/Bin**	**Batch (B)**
California Housing	ID 44025 [7]	20,640	8	0	256
House 16H	ID 44139 [7]	22,784	16	0	256
Diamonds	ID 44059 [7]	53,940	6	3	512
Elevators	ID 44137 [7]	16,599	16	0	256
Abalone	ID 44956 [7]	4,177	7	1	256
CPU Act	ID 44132 [7]	8,192	21	0	256
Ailerons	ID 44136 [7]	13,750	33	0	256
Brazilian Houses	ID 44141 [7]	10,692	8	3	256
Bike Sharing Demand	ID 44142 [7]	17,379	6	5	256
House Sales	ID 44144 [7]	21,613	15	2	256
Miami Housing 2016	ID 44146 [7]	13,932	13	0	256
Superconduct	ID 44148 [7]	21,263	79	0	256

5.3 Preprocessing Procedures

To prevent data leakage, all preprocessing statistics (imputation values, quantile parameters, and target normalization) will be fitted exclusively on the training partition relevant to each step. During tuning, preprocessing will be fitted on the inner training partition and applied to the single inner validation partition. For final refitting, preprocessing will be fitted on the full outer training partition and applied unchanged to the outer test fold. Following the standardized preprocessing protocol of the Inria Tabular Benchmark [7]:

1. **Numerical Features:** Missing numerical values are imputed using the median of the relevant training partition. Continuous features are transformed using `scikit-learn`'s `QuantileTransformer` (mapping distributions to a standard normal distribution $\mathcal{N}(0,1)$), which mitigates the impact of heavy-tailed outliers and skewed feature distributions common in tabular datasets.

2. **Categorical and Binary Features:** Missing categorical values are assigned to a dedicated `<UNK>` token. Binary features are mapped to $\{0,1\}$, while multi-category features are ordinal-encoded and passed to learnable entity embedding lookup tables within the Tabular ResNet input stem. Categories absent from the relevant training partition are routed to the `<UNK>` embedding.

3. **Regression Target Standardization:** For all regression tasks, target values y are standardized using the mean and standard deviation of the relevant training partition ($\tilde{y} = (y - \mu_{\text{train}})/\sigma_{\text{train}}$) so that MSE loss magnitudes and orthogonality regularization weights λ remain on a consistent scale across datasets. Model predictions are inverse-transformed using those same training statistics before computing RMSE and MAE.

6 Results Aggregation and Statistical Analysis

6.1 Approach to Aggregating Results

Our experimental design evaluates K model methods across $N_{\text{clf}} = 12$ classification datasets and $N_{\text{reg}} = 12$ regression datasets ($N_{\text{total}} = 24$), with each dataset evaluated using $F = 5$ outer folds and $S = 5$ fixed random seeds per outer-fold refit. Hyperparameters are selected independently within each outer training partition, so each method's selected hyperparameter values may differ across folds. We aggregate the untouched outer-test results at two levels:

1. Within-Dataset Aggregation (Capturing Accuracy and Stability). Let $m_{d,c}^{(f,s)}$ denote the outer-test metric of method $c \in \{1,\dots,K\}$ on dataset d, outer fold $f \in \{1,\dots,F\}$, and random seed $s \in \{1,\dots,S\}$. First, we compute the fold-level mean across seeds, $\bar{m}_{d,c}^{(f)} = \frac{1}{S}\sum_{s=1}^{S} m_{d,c}^{(f,s)}$, and the within-fold seed standard deviation, $s_{d,c}^{(f)} = \sqrt{\frac{1}{S-1}\sum_{s=1}^{S}\left(m_{d,c}^{(f,s)} - \bar{m}_{d,c}^{(f)}\right)^2}$. For every dataset–method pair, we report:

$$\hat{\mu}_{d,c} = \frac{1}{F}\sum_{f=1}^{F} \bar{m}_{d,c}^{(f)}, \qquad \sigma_{\text{fold}}(d,c) = \sqrt{\frac{1}{F-1}\sum_{f=1}^{F}\left(\bar{m}_{d,c}^{(f)} - \hat{\mu}_{d,c}\right)^2}, \qquad \bar{\sigma}_{\text{seed}}(d,c) = \frac{1}{F}\sum_{f=1}^{F} s_{d,c}^{(f)}. \tag{7}$$

Reporting both σ_{fold} (sensitivity to data partitioning) and $\bar{\sigma}_{\text{seed}}$ (sensitivity to random weight initialization) allows us to evaluate whether SFOvTSGM reduces prediction variance and stabilizes convergence even on tasks where mean accuracy gains are modest.

2. Cross-Dataset Aggregation. Because raw metrics (such as RMSE, MAE, or NLL) vary in scale and baseline difficulty across datasets, direct arithmetic averaging of raw scores across datasets is dominated by high-variance tasks. We will aggregate performance across the 12 classification datasets, the 12 regression datasets, and all 24 datasets combined using three scale-invariant summaries:

- **Mean Rank ($\bar{R}_c$):** On each dataset d, the K methods are ranked from 1 (best) to K (worst) based on $\hat{\mu}_{d,c}$ (assigning average ranks in the event of ties). We report the mean rank $\bar{R}_c = \frac{1}{N_d}\sum_{d=1}^{N_d} R_{d,c}$ [3].
- **Relative Error / Loss Ratio (Geometric Mean):** For strictly positive loss metrics (NLL for classification, RMSE and MAE for regression), we normalize each model's score on dataset d by the Single Tabular ResNet baseline score, $r_{d,c} = \hat{\mu}_{d,c}/\hat{\mu}_{d,\text{Single}}$, and compute the geometric mean across datasets: $\text{GM}_c = \left(\prod_{d=1}^{N_d} r_{d,c}\right)^{1/N_d}$.
- **Pairwise Win / Tie / Loss Counts:** Summary tables tabulating how many datasets a given method improves upon, ties, or degrades relative to the Unregularized Ensemble and Single ResNet baselines.

6.2 Planned Statistical Tests

To ensure that observed differences are not artifacts of random variation, we follow the non-parametric statistical testing framework established by Demšar [3] and refined by Benavoli et al. [2]. All tests will be conducted at a significance level of $\alpha = 0.05$.

1. Omnibus Multi-Model Comparison (Friedman Test with Iman–Davenport Correction). To test the global null hypothesis H_0 that all K evaluated methods perform equally well across N_d datasets ($N_d = 12$ for task-specific analysis; $N_d = 24$ for unified rank analysis across all tasks using NLL and RMSE ranks), we compute the Friedman statistic χ_F^2 alongside the less conservative Iman–Davenport F_F statistic:

$$\chi_F^2 = \frac{12N_d}{K(K+1)}\left[\sum_{c=1}^{K}\bar{R}_c^2 - \frac{K(K+1)^2}{4}\right], \qquad F_F = \frac{(N_d-1)\chi_F^2}{N_d(K-1)-\chi_F^2}, \tag{8}$$

where F_F follows an F-distribution with $(K-1)$ and $(K-1)(N_d-1)$ degrees of freedom.

2. Post-Hoc Comparisons and Critical Difference Diagrams. If the omnibus test rejects H_0 ($p < 0.05$), we will conduct two complementary post-hoc analyses:

- **Nemenyi Post-Hoc Test and Critical Difference (CD) Diagrams [3]:** For visual all-pairs comparison, we will generate CD diagrams where methods whose average ranks differ by less than $\text{CD} = q_\alpha\sqrt{\frac{K(K+1)}{6N_d}}$ are connected by a horizontal clique bar.
- **Wilcoxon Signed-Rank Test with Holm–Bonferroni Correction [2]:** Because rank-based Nemenyi tests have lower statistical power and depend on the pool of third-party baselines, we will conduct paired two-sided **Wilcoxon Signed-Rank Tests** directly comparing our proposed SFOvTSGM ensembles against each baseline (Unregularized Ensemble, Symmetric Orthogonal Ensemble, BatchEnsemble, and TANGOS). For comparisons on a single metric across $N_d = 12$ datasets, we use raw paired differences $\Delta_d = \hat{\mu}_{d,A} - \hat{\mu}_{d,B}$; for combined $N_d = 24$ tests, we use relative loss ratios $r_{d,c}$. To control the Family-Wise Error Rate (FWER) across multiple pairwise comparisons, p-values will be adjusted using the step-down **Holm–Bonferroni procedure**.

3. Within-Dataset Hypothesis Testing. Because repeating experiments across random seeds within a dataset does not by itself establish statistical significance, we will test whether performance differences on each individual dataset d are statistically distinguishable from noise. Specifically, we will apply a paired two-sided test across the fold-level scores $\bar{m}_{d,c}^{(f)}$, marking statistically significant improvements over the Unregularized Ensemble baseline in bold in all per-dataset result tables.

7 Reproducibility and Declarations

7.1 Reproducibility

To guarantee full experimental reproducibility across hardware environments, the project will enforce strict determinism and standardized code distribution:

1. **Deterministic Random Seeding:** All sources of stochasticity—including dataset cross-validation partitioning, weight initialization, dropout masks, mini-batch shuffling, and Bayesian hyperparameter sampling—will be controlled using fixed integer seeds:
   - Cross-validation splits (`StratifiedKFold` and `KFold` in `scikit-learn`) and `Optuna`'s `TPESampler` will be initialized with a fixed master seed ($s_{\text{master}} = 42$).
   - For the $S = 5$ repeated evaluation runs per fold, all random number generators (`python.random`, `numpy.random`, `torch.manual_seed`, and `torch.cuda.manual_seed_all`) will be explicitly initialized with the fixed seed set $\mathcal{S} = \{42, 67, 69, 228, 2137\}$.
   - Within each M-model ensemble run initialized with global seed $s \in \mathcal{S}$, individual member networks f_m ($m \in \{1, \ldots, M\}$) will be initialized deterministically using derived sub-seeds $s_m = 1000s + m$ so that starting weights are distinct across learners yet strictly reproducible across runs.
   - PyTorch deterministic execution flags (`torch.backends.cudnn.deterministic = True` and `torch.backends.cudnn.benchmark = False`) will be enabled during training and evaluation.
2. **Public Repository and Execution Instructions:** All source code, preprocessing pipelines, tuned hyperparameter configuration files, and statistical analysis scripts will be maintained in a public GitHub repository (and included in the final LeON submission archive). The repository will contain:
   - A pinned environment specification (`requirements.txt` and `environment.yml`) locking exact versions of Python, PyTorch, CUDA, `scikit-learn`, `openml`, and `optuna`.
   - Automated data-fetching scripts that download and cache the 24 OpenML datasets via their exact dataset IDs (Tables 2 and 3).
   - A step-by-step `README.md` documenting the exact command-line instructions required to reproduce hyperparameter tuning, model training, metric aggregation, and figure/table generation from scratch.

7.2 External Code and AI Usage Disclosure

Planned Use and Modification of External Code. In accordance with course guidelines, any external code utilized for baselines or data loading will be explicitly cited in the final report and substantially modified to fit our experimental pipeline:

- **Data Loading and Tabular ResNet Reference:** We will adapt dataset loading utilities and reference Tabular ResNet block definitions from the open-source Inria Tabular

Benchmark repository (`LeoGrin/tabular-benchmark`) [7], modifying the architecture to expose penultimate batch-centered latent representations $\tilde{Z}_m$ and support simultaneous multi-model forward passes.

- **TANGOS Baseline:** We will adapt the intra-model gradient orthogonalization loss calculation from the official Cambridge repository (`vanderschaarlab/TANGOS`) [9], integrating it into our unified Tabular ResNet training loop.
- **Original Implementation:** The core contributions of this project—specifically the additive multi-network ensemble wrapper, the lower-triangular stop-gradient mask (SFOvTSGM), and the vectorized Φ_{CKA} and Φ_{Cos} repulsion loss modules—will be implemented from scratch in PyTorch.

AI Usage Disclosure. In compliance with the course policy on artificial intelligence disclosure, the authors declare that the text, mathematical typesetting, and LaTeX structure of this Project Plan (spanning Sections 1 through 7 and the accompanying BibTeX bibliography) were generated in their entirety by a Large Language Model (AI assistant).

This generation process was conducted under continuous, substantial human supervision and direction. fir

References

[1] Sercan Ö Arik and Tomas Pfister. TabNet: Attentive interpretable tabular learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 6679–6687, 2021.

[2] Alessio Benavoli, Giorgio Corani, and Francesca Mangili. Should we really use post-hoc tests based on mean-ranks? *Journal of Machine Learning Research*, 17(5):1–10, 2016.

[3] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[4] Zhijie Deng, Jiaxin Shi, and Jun Zhu. NeuralEF: Deconstructing kernels by deep neural networks. In *International Conference on Machine Learning (ICML)*, pages 4976–4992. PMLR, 2022.

[5] Stanislav Fort, Huiyi Hu, and Balaji Lakshminarayanan. Deep ensembles: A loss landscape perspective. *arXiv preprint arXiv:1912.02757*, 2019.

[6] Ian Gemp, Brian McWilliams, Claire Vernade, and Thore Graepel. EigenGame: PCA as a Nash equilibrium. In *International Conference on Learning Representations (ICLR)*, 2021.

[7] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, volume 35, pages 507–520, 2022.

[8] Yi Gu, Yi Lin, Kwang-Ting Cheng, and Hao Chen. Revisiting deep ensemble uncertainty for enhanced medical anomaly detection. In *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, 2024.

[9] Alan Jeffares, Tennison Liu, Jonathan Crabbé, Fergus Imrie, and Mihaela van der Schaar. TANGOS: Regularizing tabular neural networks through gradient orthogonalization and specialization. In *International Conference on Learning Representations (ICLR)*, 2023.

[10] Alan Jeffares, Tennison Liu, Jonathan Crabbé, and Mihaela van der Schaar. Joint training of deep ensembles fails due to learner collusion. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[11] Arlind Kadra, Marius Lindauer, Frank Hutter, and Josif Grabocka. Well-tuned simple nets excel on tabular datasets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 23928–23941, 2021.

[12] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *International Conference on Machine Learning (ICML)*, pages 3519–3529. PMLR, 2019.

[13] Agostina J Larrazabal, Cesar Martinez, Jose Dolz, and Enzo Ferrante. Orthogonal ensemble networks for biomedical image segmentation. In *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, pages 594–603. Springer, 2021.

[14] Yong Liu and Xin Yao. Ensemble learning via negative correlation. *Neural Networks*, 12(10):1399–1404, 1999.

[15] Yeming Wen, Dustin Tran, and Jimmy Ba. BatchEnsemble: An alternative approach to efficient ensemble and lifelong learning. In *International Conference on Learning Representations (ICLR)*, 2020.